

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA02

COURSE NAME: C Programming

COURSE OUTCOME COVERED

CO2: Examine and write solutions for problems using operators, decision-making, and looping constructs. (BL3,BL4)

Assessment Tool Weightage: 40%

QUESTIONS:

Total Marks:100

Annexure A

Saveetha Compiler assessment (Level1& Viva Voce)

Code of Conduct:

I Kavya - 192425283 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Kavya

Signature of the student:

90/100

85

LEVEL 1 EXAMINATION

Digit ▸

Programs:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int n, count=0;
```

```
printf("Enter a number:");
```

```
scanf("%d", &n);
```

```
while (n!=0) {
```

```
    n=n/10;
```

```
    count++;
```

```
}
```

```
printf("Number of digits = %d", count);
```

```
return;
```

```
}
```

Input:

12345

Output:

5

Result:

the program executed successfully

Octal ▸

programs:

```
#include <stdio.h>
```

```
int main() {
```

```
int n, octal=0, place=1, rem;
```

```
printf("Enter a decimal number: ");
```

```
scanf("%d", &n);
```

```
while (n != 0) {
```

```
    rem = n % 8;
```

```
    octal = octal + rem * place;
```

```
    place = place * 10;
```

```
    n = n / 8;
```

```
}
```

```
printf("Octal equivalent = %d", octal);
```

```
return 0;
```

```
}
```

Input :

Enter a decimal number = 65

Output :

Octal equivalent = 101

Result :

The program executed successfully

prime number
#include <stdio.h>
int main()

{ int a, b, i, j;

scanf ("%d %d", &a, &b);

for (i = a; i <= b; i++) {

int prime = 1;

if (i < 2)

prime = 0;

for (j = 2; j < i; j++) {

if (i % j == 0) {

prime = 0;

break;

}
if (prime)

printf ("%d", i);

return 0;

Input

10 and 20

Output

11, 13, 17, 19.

Result:

The program executed successfully

Hollow pyramid

Program:

```
#include <stdio.h>
```

```
int main()
```

```
{  
    int i, j;
```

```
    for (i = 1; i <= 4; i++)
```

```
    {  
        for (j = 1; j <= 4 - i; j++)
```

```
            printf(" ");
```

```
        for (j = 1; j <= 2 * i - 1; j++)
```

```
        {  
            if (i == 4 || j == 1 || j == 2 * i - 1)
```

```
                printf("*");
```

```
            else
```

```
                printf(" ");  
        }
```

input and output

```
* *  
  
x *  
  
* *  
  
x x * x x x
```

Result:

thus the program executed successfully

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Logic Development	35%	Develops correct, efficient, and well-structured logic for the problem	Logic is mostly correct with minor gaps	Logic is partially correct; needs improvement	Incorrect or no logical approach
Syntax & Semantics	20%	Writes error-free code with correct syntax and semantics	Minor syntax/semantic errors that do not affect execution significantly	Multiple errors; requires corrections	Frequent errors; code does not compile/execute
Time & Space Complexity	20%	Demonstrates optimal time and space complexity with proper justification	Reasonably efficient with minor scope for improvement	Limited consideration of complexity	No consideration of time or space complexity
Test Case Validation	20%	Designs and validates comprehensive test cases including edge cases	Tests cover most scenarios with minor gaps	Limited test cases; misses important cases	No or incorrect test case validation
Viva Voce	15%	Explains logic, code, and concepts clearly with confidence and depth	Explains concepts with minor gaps	Limited understanding; needs prompting	Unable to explain concepts